3D Data Science with Python

Sergio Peral · ML Engineer, Teqyard



3D used to mean C++. Not anymore

C++ with PCL

```
using Cloud =
    pcl::PointCloud<pcl::PointXYZ>;
Cloud::Ptr cloud(new Cloud);
Cloud down;
if (pcl::io::loadPCDFile(
    "crop.pcd", *cloud)) return 1;
pcl::VoxelGrid<pcl::PointXYZ> vg;
vg.setInputCloud(cloud);
vg.setLeafSize(0.5f, 0.5f, 0.5f);
vg.filter(down);
std::cout << cloud->size()
  << " -> " << down.size() << "\n";


# output
722961 -> 230519
```

Python with Open3D

```
pcd = o3d.io.read_point_cloud(path)
down = pcd.voxel_down_sample(0.5)
print(len(pcd.points), "->",
      len(down.points))


# output
722961 -> 232180
```

Why the counts differ: both average the points in each 0.5 m voxel, but the grids sit in different places. PCL anchors its voxels at multiples of the leaf size; Open3D at the minimum corner minus half a voxel.

Python is the interface. C++ does the work



Every call you make here runs compiled code

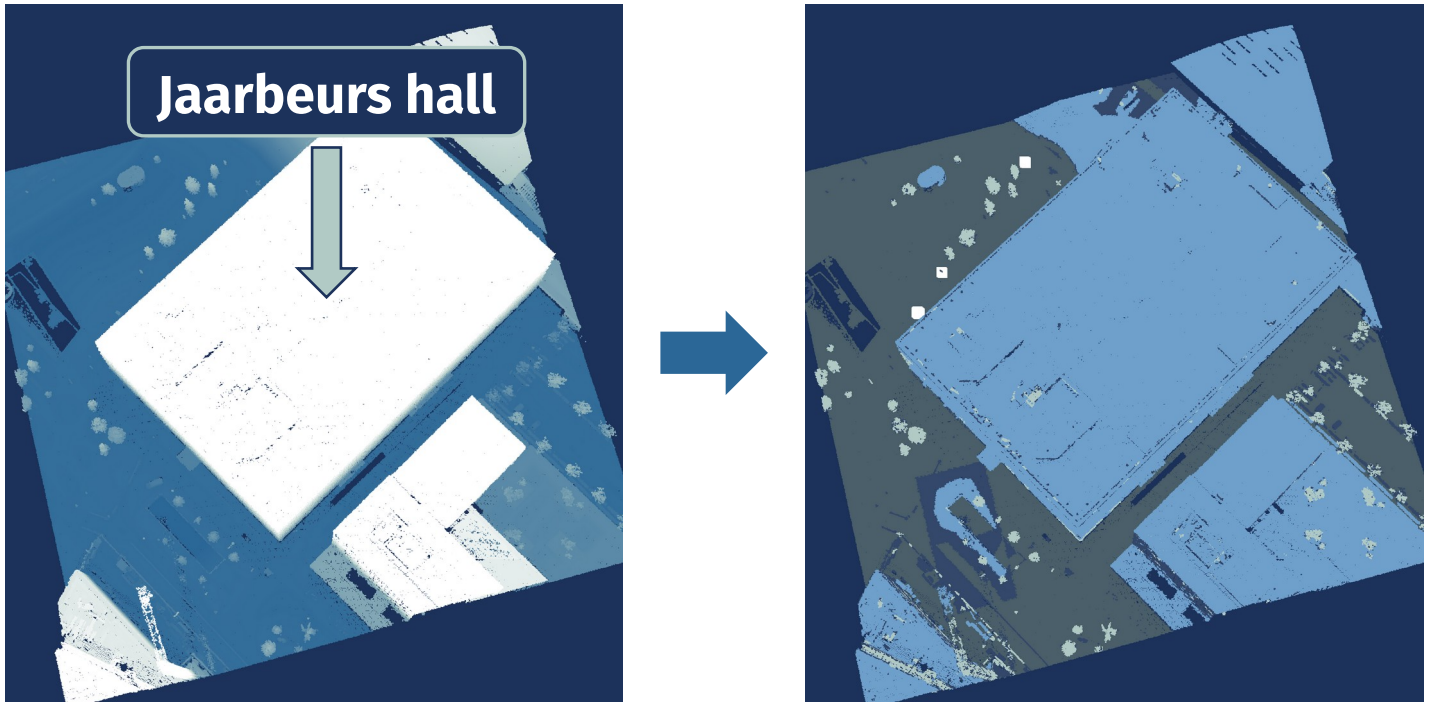
- NumPy: a compiled C core, with some C++
- Open3D: C++, wrapped for Python
- PyVista: VTK, which is C++
- laspy: the lazrs decoder is Rust

Python is used as a proxy to write higher-level code: the heavy loops run below it, in compiled code.

The Python 3D ecosystem

Load and store	laspy	PDAL	pyproj	
Arrays and speed	NumPy	SciPy	Numba	dask
Process	OPEN3D	trimesh		
Visualise	PyVista	VTK		
Learn	scikit-learn	PyTorch3D		

Today's case: the Jaarbeurs hall, from a plane



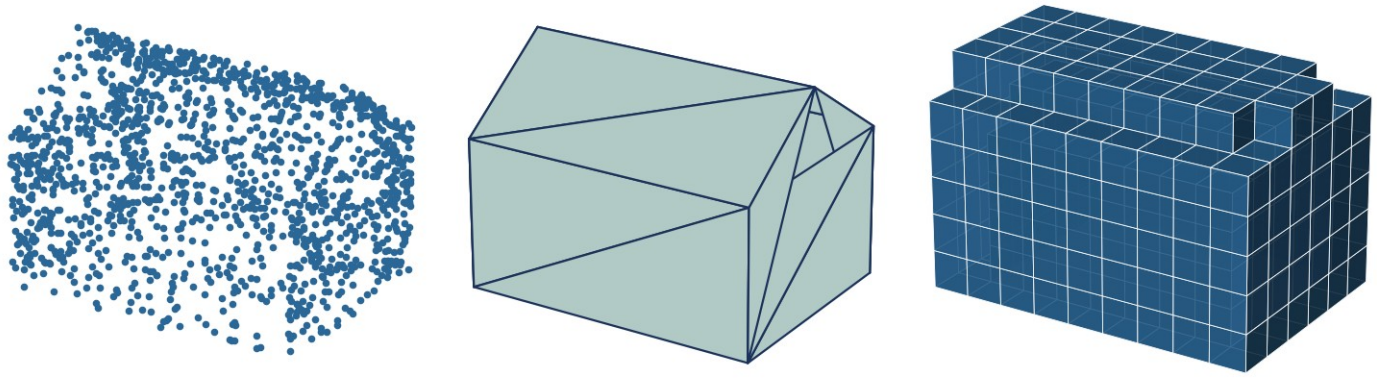
The data set

Area	200 m × 200 m
Points	722 961
Density	about 18 per m²
Source	AHN5, ahn.nl
Flown	April 2023, night
File	LAZ, 4.3 MB
Coordinates	RD New + NAP
Licence	CC BY 4.0

Goal: ground, buildings, vegetation, street furniture; scored against AHN.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

One object, three representations



Point cloud

1500 × 3 floats
no connectivity

Mesh

10 vertices, 16 triangles
a surface with neighbours

Voxel grid

182 occupied 0.5 m cells
volume at one resolution

Points measure, meshes connect, voxels index space. Convert only when the task needs it.

- Point cloud: an unordered N×3 array plus attributes; no neighbours stored.
- Mesh: vertices plus faces, so surfaces, normals and areas are defined.
- Voxel grid: occupied cells at one resolution; detail below a cell is gone.

A 3D project moves through six stages



Acquisition
Scan the scene
laspy

1

2

Preparation
Clean and register
NumPy, Open3D

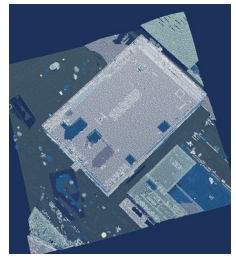
Application
Ship it

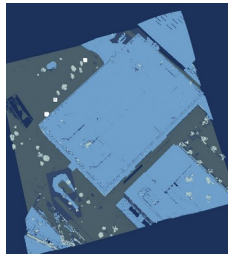
6

Loop back when the data surprises

3

Exploration
Model and label
Open3D, SciPy





Visualisation
See it, trust it
PyVista

5

4

Analysis
Measure, monitor
NumPy, SciPy

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

Four rules of thumb for 3D data

1 Think in arrays, not points

A point cloud is an N×3 table plus attribute columns.

2 Look before you compute

Render it; check units, coordinates and classes.

3 Make it small first

Crop, tile and downsample; scale up last.

4 Let compiled code loop

One call per array or tile, never per point.

No single library does the whole job

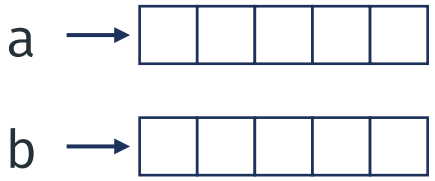
Load	Voxel downsampling	Ground removal	Clustering	Labelling	Visualisation
laspy LAS/LAZ in and out	**Open3D** downsampling, planes, DBSCAN				**PyVista** off-screen renders
			SciPy KDTree batch neighbour queries		
NumPy the arrays every step passes on					

Use views, not copies

Why 24 B? x, y and z are three float64 values: 3 × 8 B.

Every unnecessary xyz copy costs 24 B per point → 0.5 GB per 1 km AHN file.

The problem: copies **copy**



new memory → costs time & RAM

The solution: views **view**

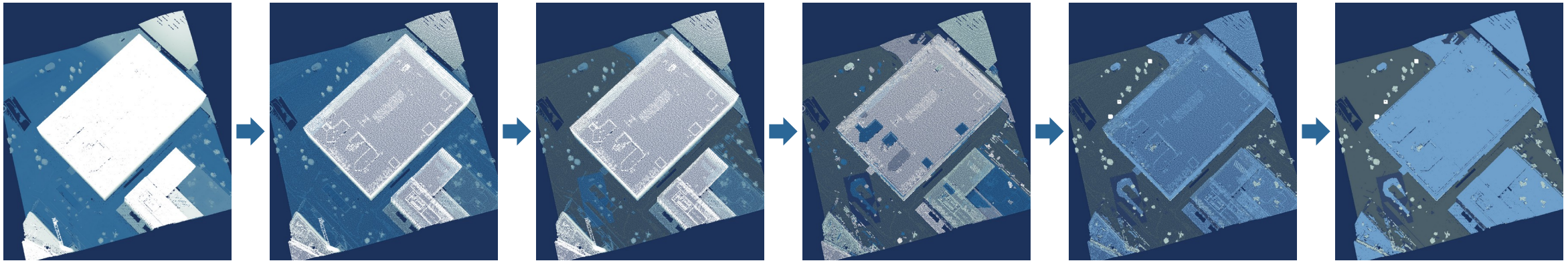


same memory → free, writes show through

`las.xyz`	**copy**	new array, every access
`Vector3dVector(xyz)`	**copy**	float32: 39× slower
`np.asarray(pcd.points)`	**view**	writes reach the point cloud
`pv.PolyData(pts)`	**view**	same memory as pts

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/python_facts.json.

Six steps turn a point cloud into labels



Load	**Voxel downsampling**	**Ground removal**	**Clustering**	**Labelling**	**Visualisation**
laspy	Open3D	Open3D	Open3D, SciPy	NumPy, SciPy	SciPy, PyVista

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

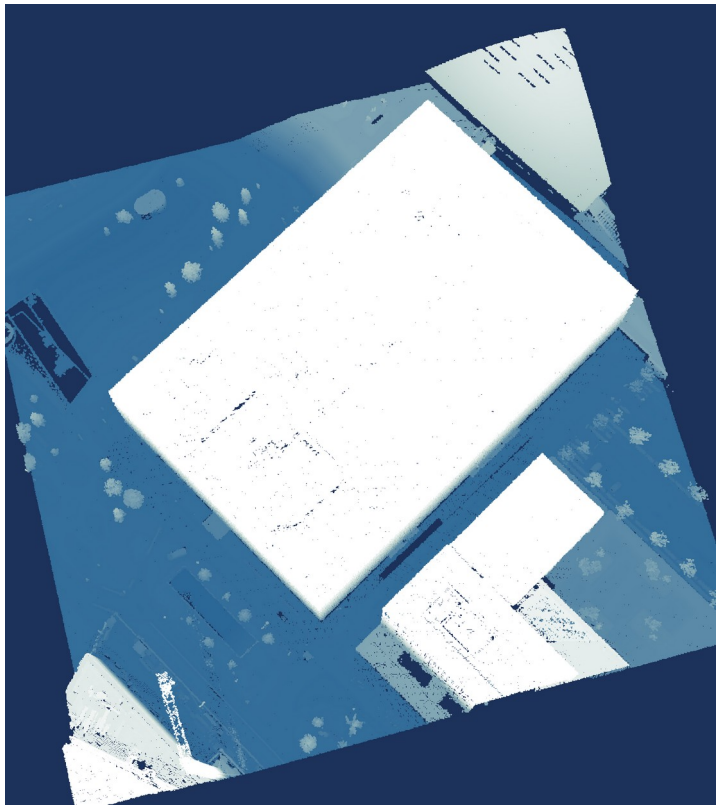
1 Load: laspy hands you NumPy

```python
import laspy
import open3d as o3d
las = laspy.read("crop.laz")
xyz = las.xyz - las.header.mins
v = o3d.utility.Vector3dVector(xyz)
pcd = o3d.geometry.PointCloud(v)
```



float32 step

1.6 cm →
after origin shift:
15 µm

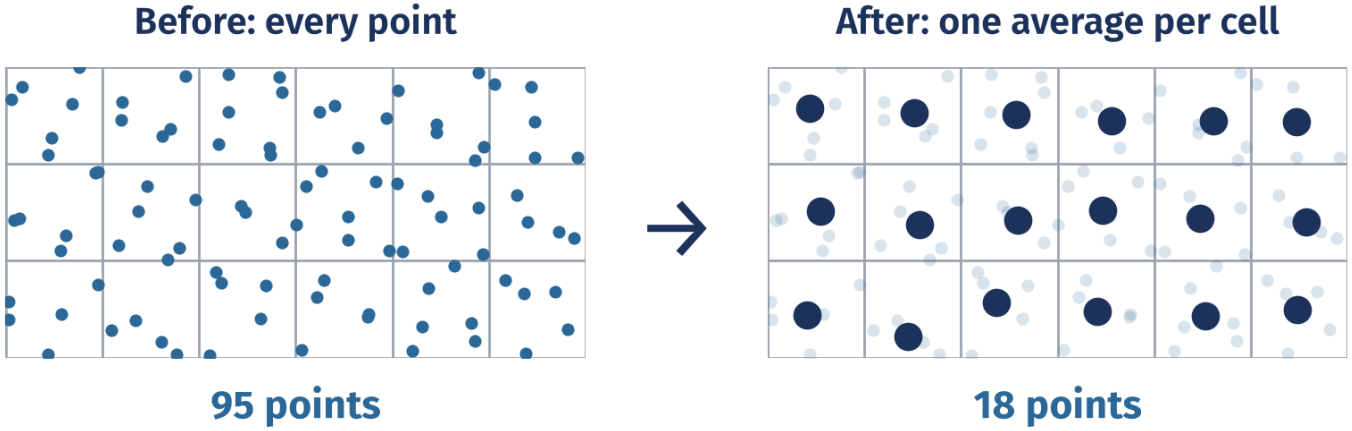


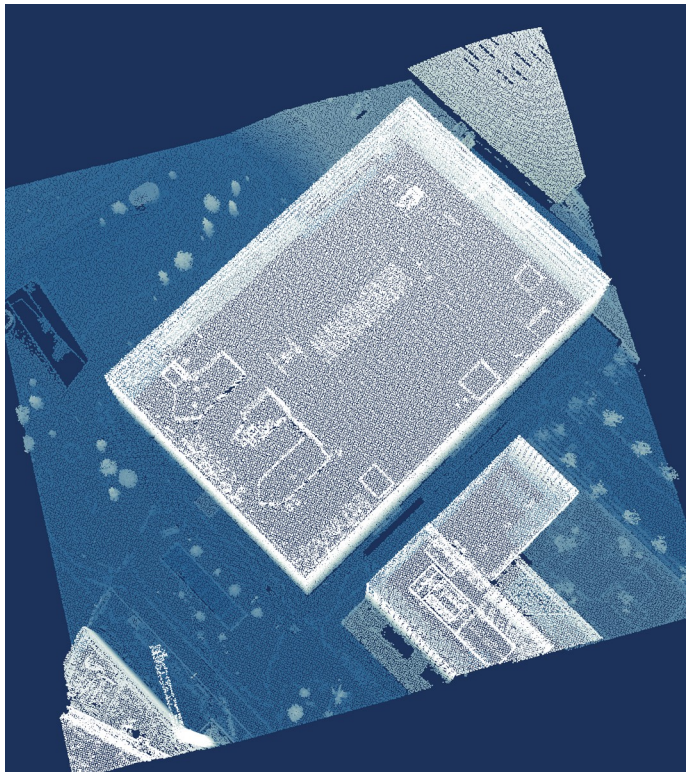
Subtract a local origin before anything else.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

2 Voxel downsampling: one average per voxel

```
down = pcd.voxel_down_sample(0.5)
P = np.asarray(down.points)
print(len(xyz), "->", len(P))
```





722 961 points in → 232 180 out at 0.5 m voxels.

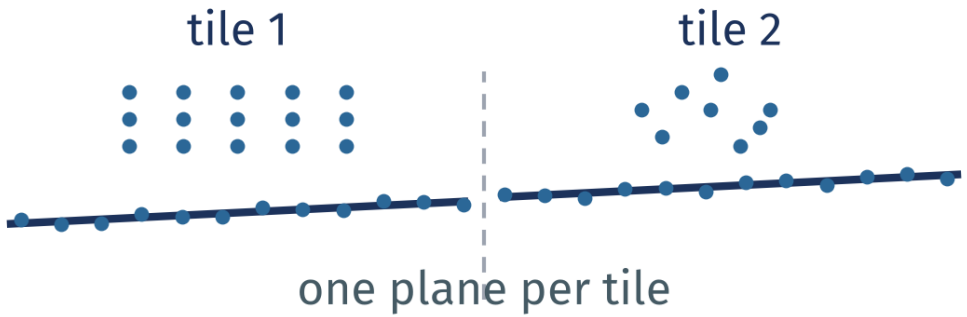
Data: AHN5, Het Waterschapshuis, CC BY 4.0.

3 Ground removal: one plane per tile

```
o3d.utility.random.seed(0)
ground = np.zeros(len(P), bool)
for i in tiles(P, 25):
    z0 = np.percentile(P[i, 2], 2)
    low = i[P[i, 2] < z0 + 1.5]
    t = down.select_by_index(low)
    _, inl = t.segment_plane(
        0.4, 3, 300,
        probability=1.0)
    ground[low[inl]] = z0 < 5
```



Scored against AHN's ground: 98 % precision, 94 % recall.

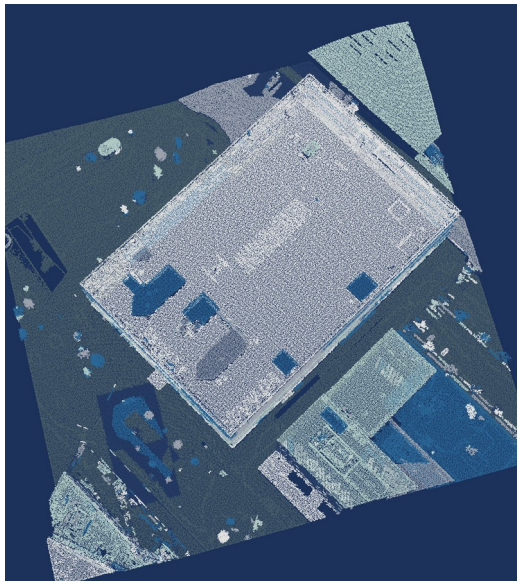


Fit one plane per tile; roofs get none.

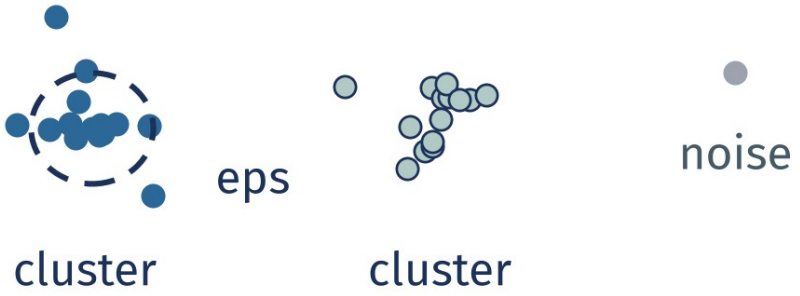
Data: AHN5, Het Waterschapshuis, CC BY 4.0.

4 Clustering: DBSCAN groups what touches

```
hag = height_above(P, ground)
up = ~ground & (hag > 0.3)
rest = down.select_by_index(
    np.flatnonzero(up))
lab = rest.cluster_dbscan(
    eps=0.75, min_points=6)
ids = np.asarray(lab)
print(ids.max() + 1, "clusters")
```



1 024 clusters.
Clusters are not labels yet.

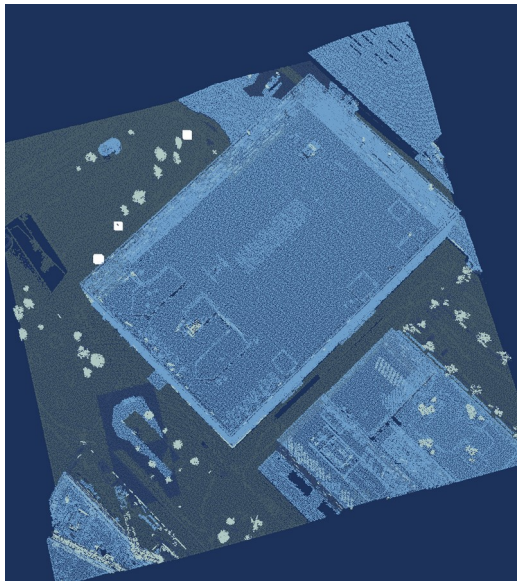


Neighbours within eps make a cluster.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

5 Labelling: rules name the clusters

```
def name(c):
    if c.multi > 0.4 and c.top > 1:
        return VEGETATION
    if c.foot > 25 and c.top > 2.5:
        return BUILDINGS
    if c.to_building < 2:  # facade
        return BUILDINGS
    if c.slender and c.free:
        return STREET_FURNITURE
    return UNLABELLED
```



Leaf-off scan: bare trees echo several times.

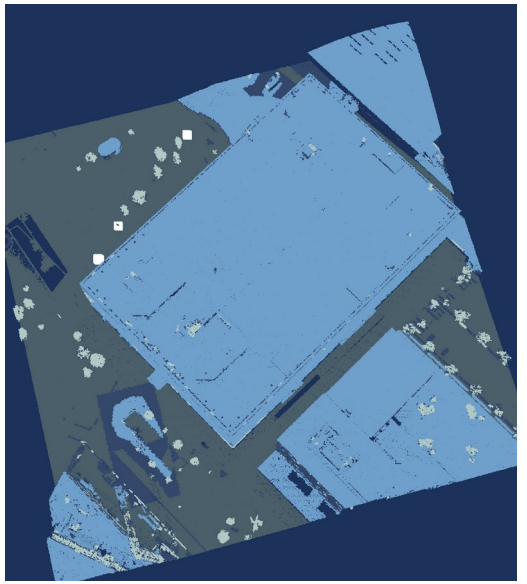


The first rule that fires names the cluster.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

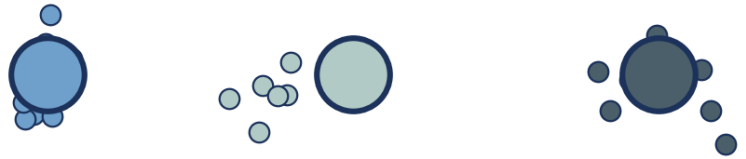
6 Visualisation: PyVista paints every point

```
P = np.asarray(down.points)
_, j = KDTree(P).query(
    xyz, workers=-1)
pl = pv.Plotter(off_screen=True)
pl.add_points(xyz, point_size=3,
    scalars=RGB[cls[j]], rgb=True)
pl.camera_position = CAM
pl.screenshot("f6_final.png")
```



Fixed camera, off-screen, one PNG per step.

Paint the labels back onto every point.

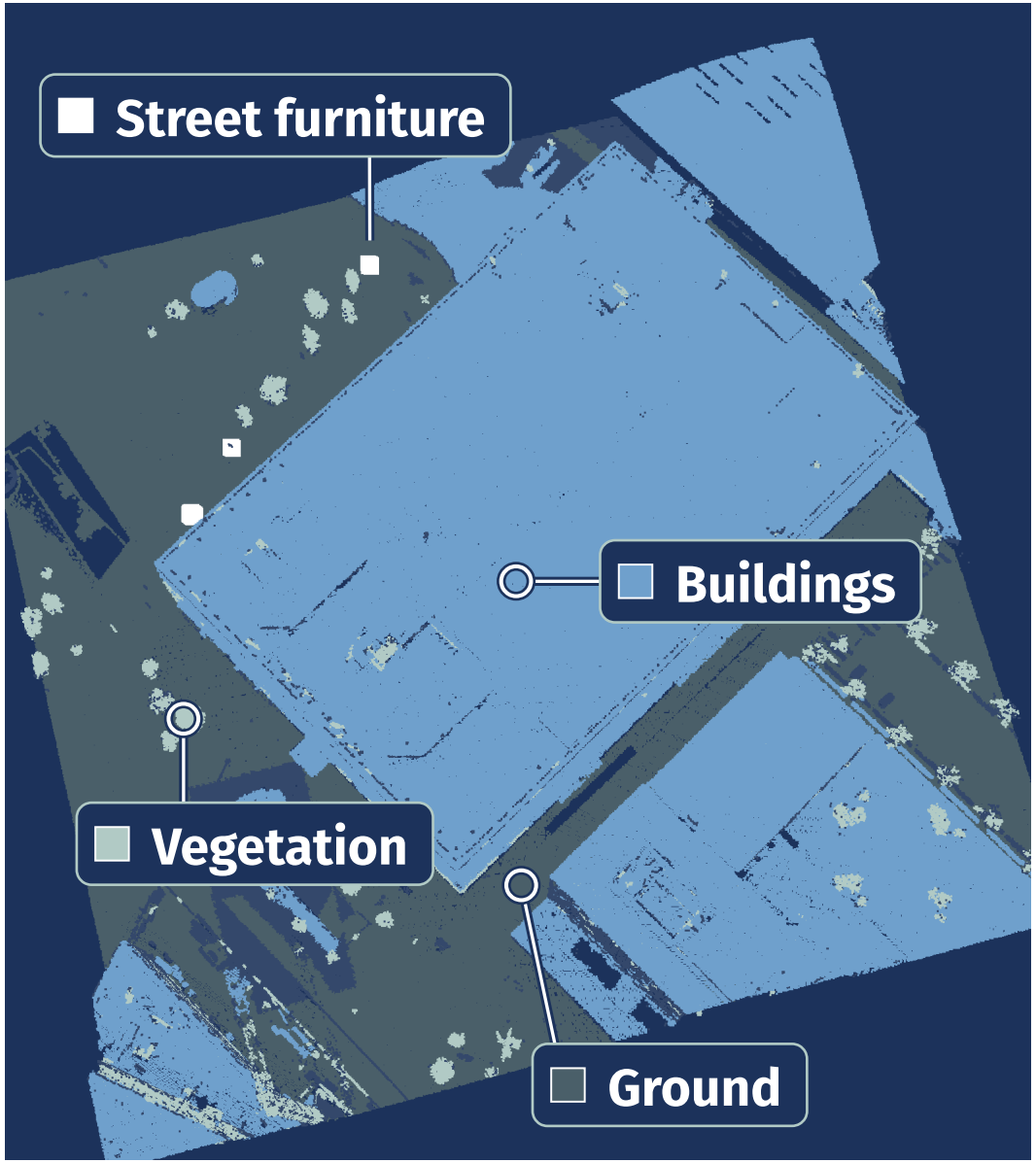


each point takes its nearest voxel's colour

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

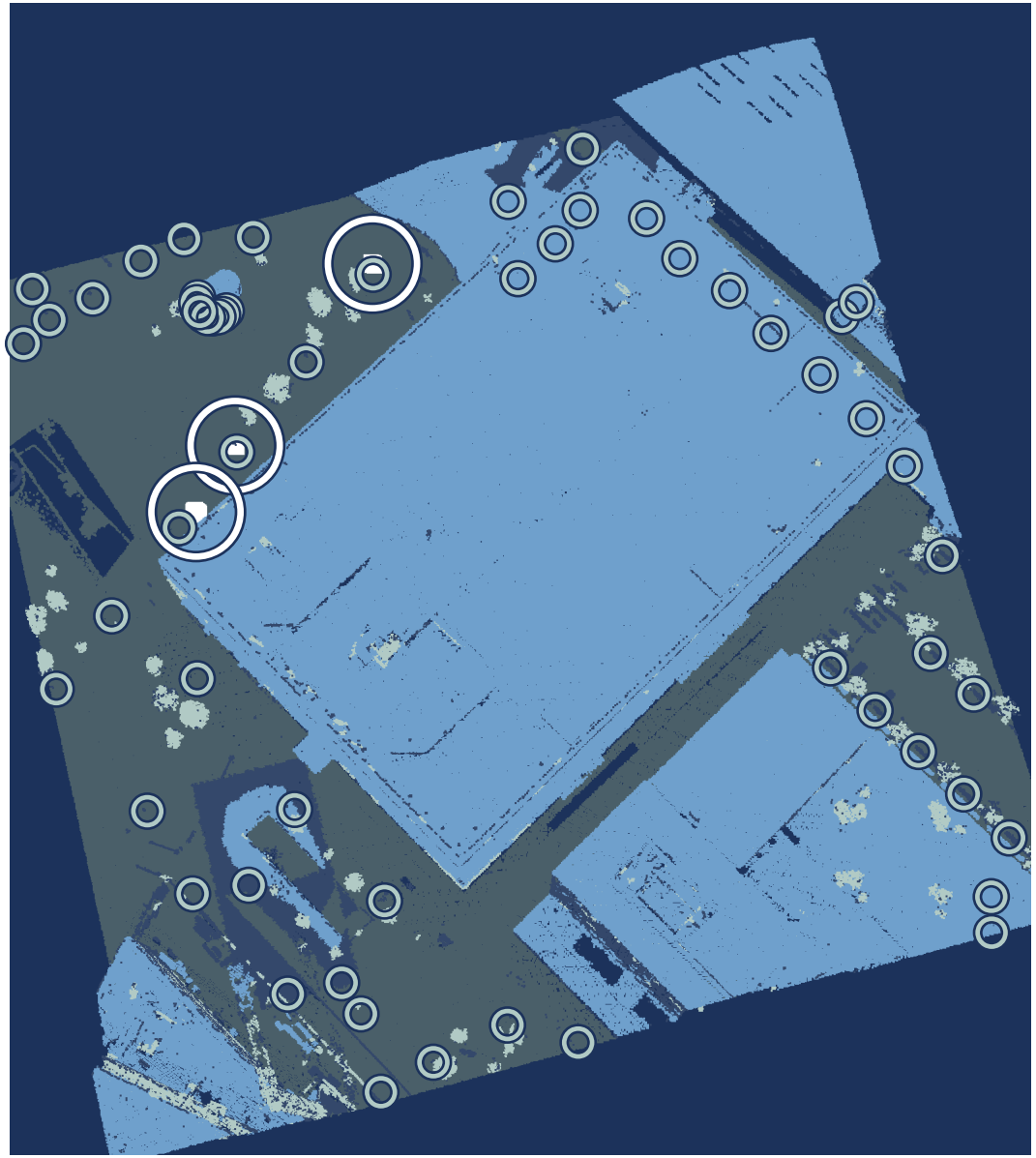
What the six steps did

Step	Library	Result	Time
Load	laspy	722 961 points	57 ms
Voxels	Open3D	232 180 voxels	84 ms
Ground	Open3D	98 % precision	81 ms
Clusters	Open3D	1 024 clusters	137 ms
Labels	NumPy	4 classes	151 ms
Paint-back	SciPy	all points	99 ms



0.6 s from file to labels and no loop over points.

Data: AHN5, Het Waterschapshuis, CC BY 4.0



Expectation vs reality: street furniture

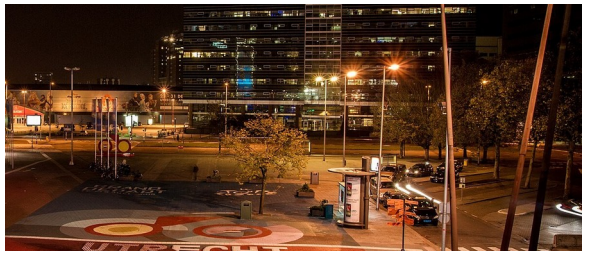
Expectation: the BGT register

61 poles 51 lamp posts, 10 bollards

Reality: our pipeline

3 objects 2 of them on a lamp post



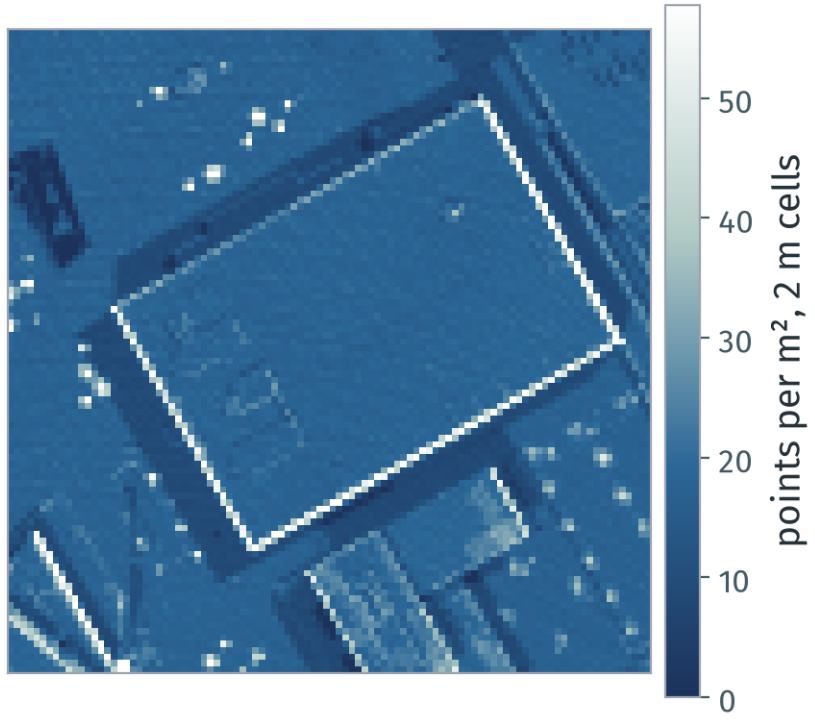


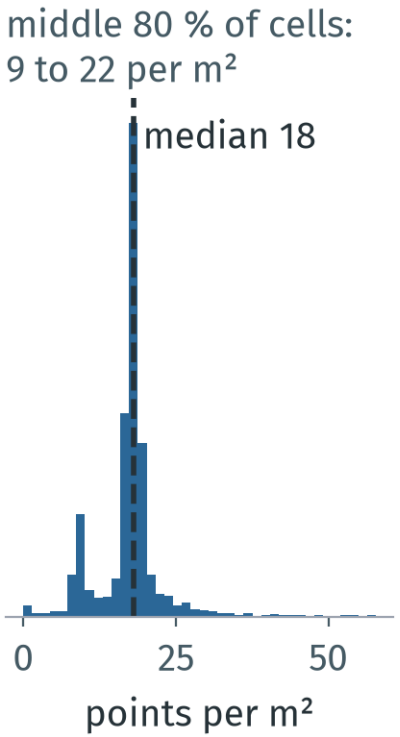
42 of 61 poles get no LiDAR point above 1.5 m.

Poles: BGT, Kadaster, CC BY 4.0. Photos: Wikimedia Commons, CC BY-SA 4.0 and CC BY 3.0.

Data: AHN5, Het Waterschapshuis, CC BY 4.0.

A point cloud samples the surface unevenly





Density, noise and gaps vary within one file. Measure them before tuning anything.

- Here the middle 80 % of cells hold 9 to 22 points per m²; median 18.
- Occlusion, flight-strip overlap, angle and material decide where points land.
- Every point carries error: AHN4 specifies at most 5 cm in height; AHN5 publishes no figure yet.

Quiz: which is faster, and by how much?

Distance of every point to the ground plane, 722 961 points.

A

```
a, b, c, d = plane
h = np.empty(len(P))  # distances
for i, p in enumerate(P):
    x, y, z = p[0], p[1], p[2]
    h[i] = abs(a*x + b*y + c*z + d)
```

B

```
a, b, c, d = plane
x, y, z = P.T  # columns, no copy
h = np.abs(a*x + b*y + c*z + d)
```

The faster one wins by about: 2× 10× 100× 1000×

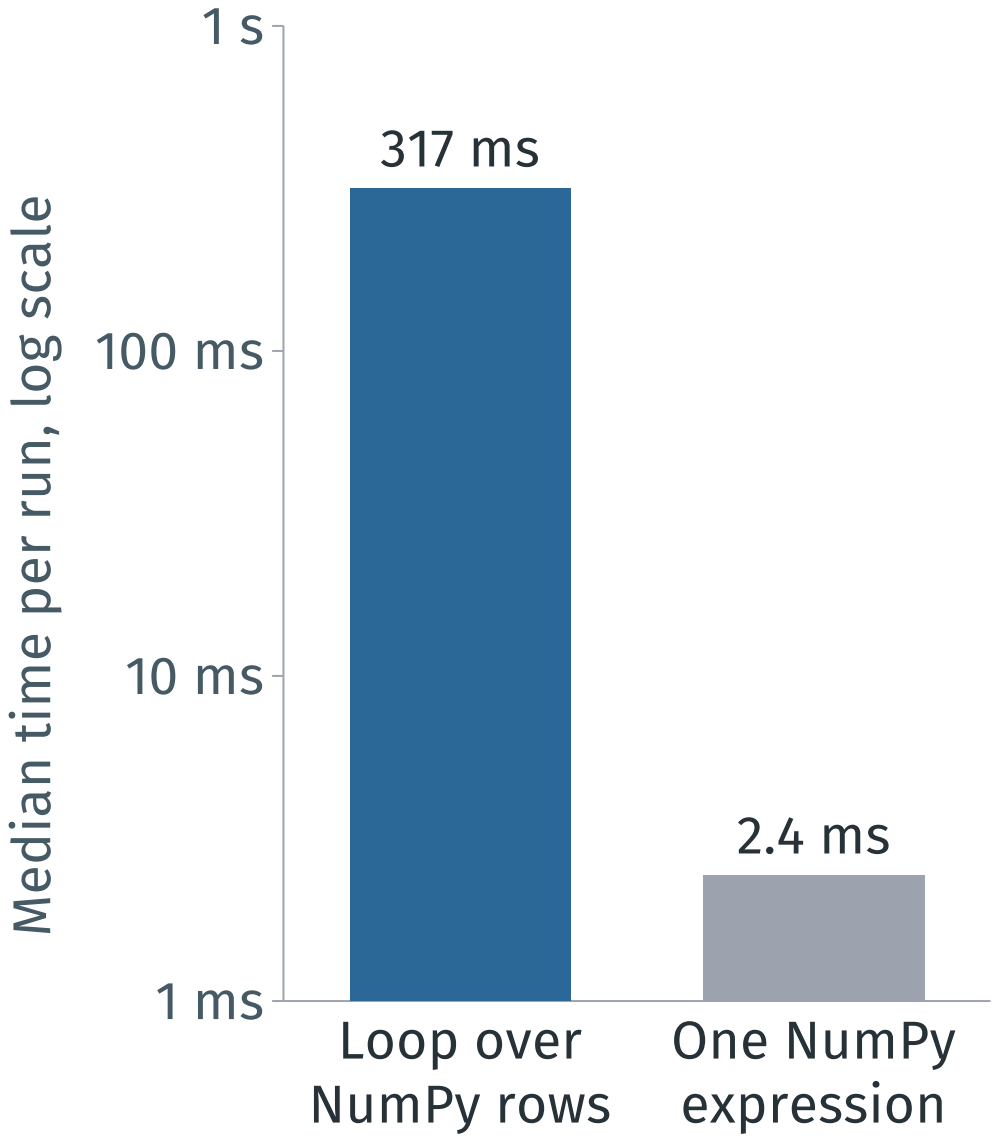
Both give identical results (np.array_equal). P: the crop's xyz from slide 12, column-ordered. plane: one seeded segment_plane on the whole crop.

How much slower is the loop over rows?

130× slower

Loop: 12 calls × 722 961 rows = 8.7 million calls

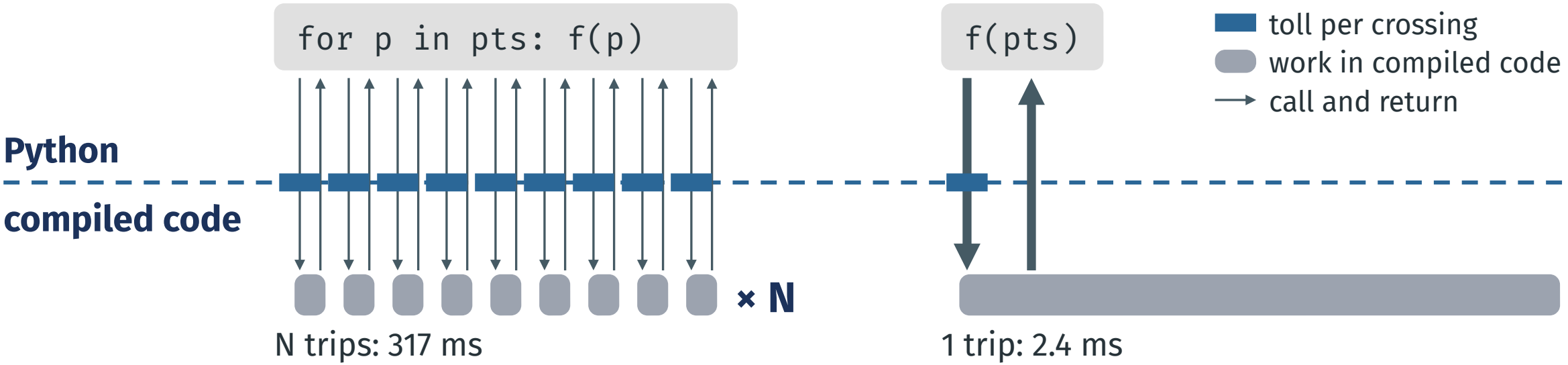
One expression: 7 calls in total



Same data, same answer, far fewer calls into compiled code.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_cliff.json.

Every trip into compiled code pays a toll



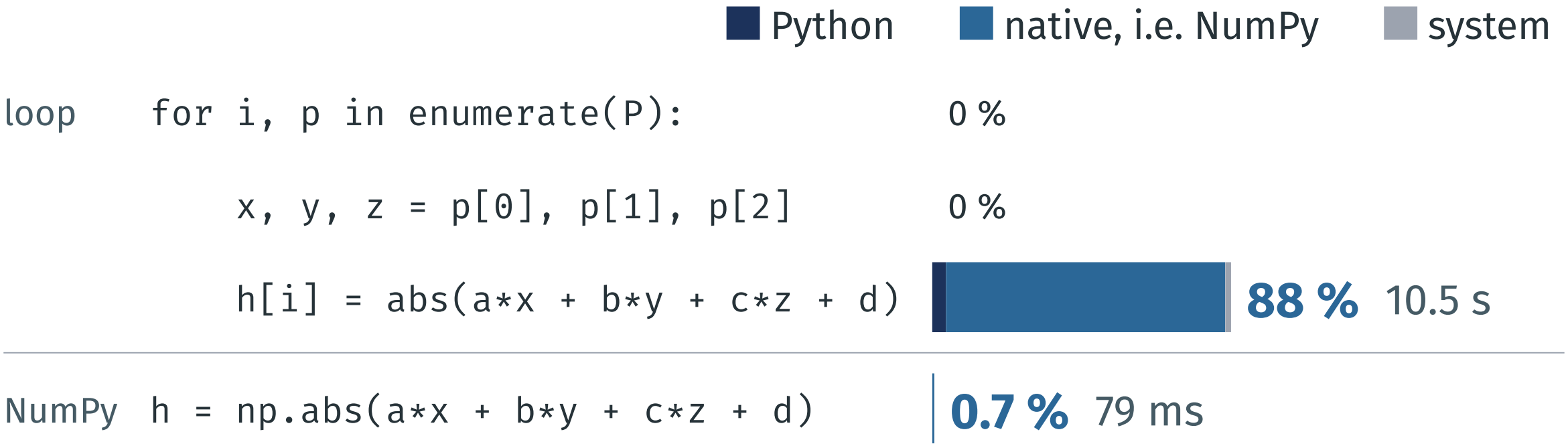
439 ns per loop step

Prefer one call on the whole array over N calls on single points.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_cliff.json.

Profile before you vectorise

```
$ scalene run --cpu-only loop.py
```



Conditions: Ryzen 7 5700X, Python 3.13.15, Scalene 2.3.0 (--cpu-only, a sample every 1 ms); 722 961 points, both versions 30 times per run, median of 3 runs; AHN5 crop (CC BY 4.0). Shares are of the whole run, imports and reading the file included; times are share × the 11.9 s run. Full: results/profile_scalene.json.

Will free-threaded Python save my loop?

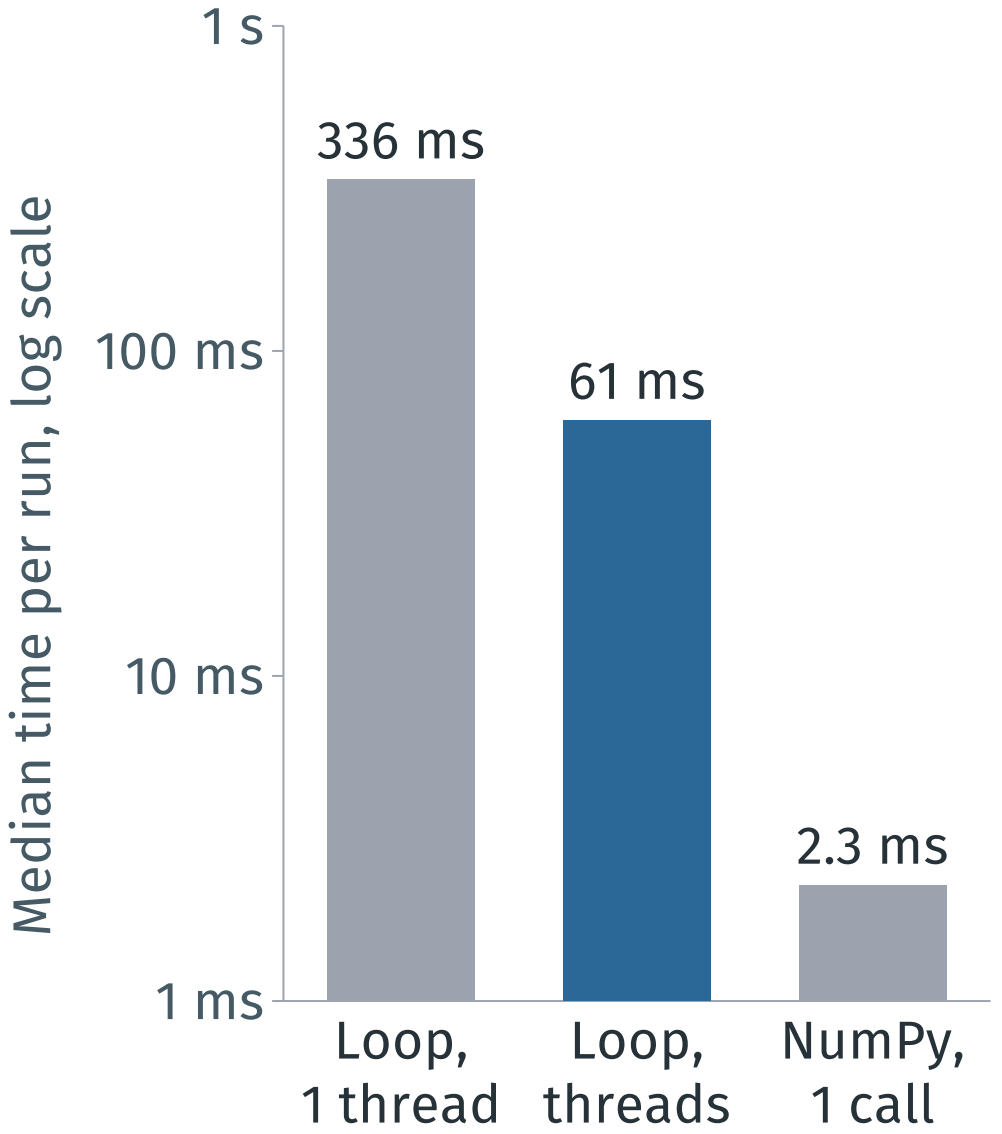
5.5× faster on 8 free threads

Loop: 12 calls × 722 961 rows = 8.7 million calls

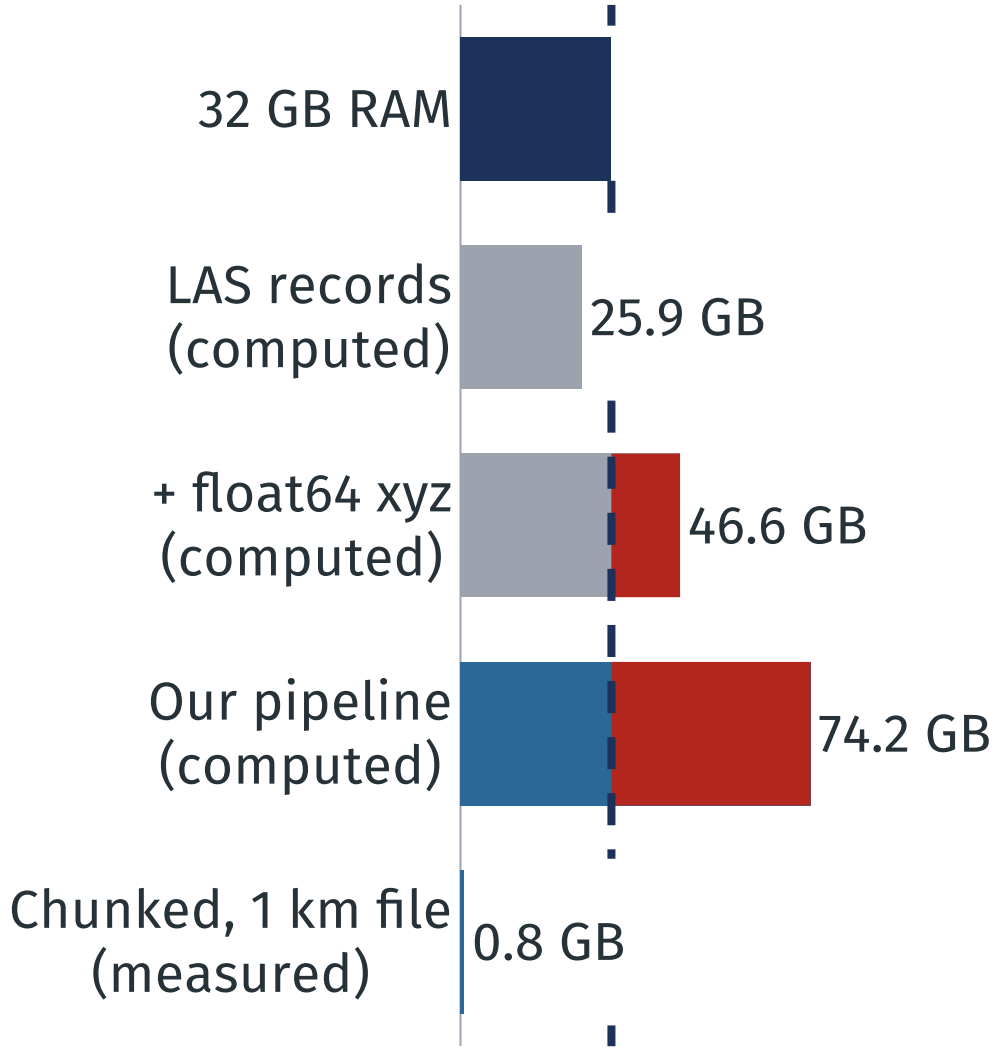
8 threads: same calls, 1.1 million each

NumPy: 7 calls in total

With the GIL, threads gain nothing; one NumPy call is still 27× faster.



Conditions: Ryzen 7 5700X, CPython 3.14.7 free-threaded and 3.14.7, NumPy 2.5.4; 722 961 points, median of 7 runs; AHN5 crop (CC BY 4.0). Full: results/bench_freethreaded.json.



Loading this area's AHN file outgrows 32 GB

86 B per point in our pipeline

926 203 338 points × 86 B each.

Why not 24 B? Each point is held several times: LAS record 30 B + float64 xyz 24 B + Open3D copy 24 B + the downsampled cloud and the rest.

It works on my machine (31.3 GiB RAM): I stream the file in chunks and keep one point per voxel.

Conditions: Ryzen 7 5700X, Python 3.13.15, NumPy 2.5.3; 21 013 926 points, median of 3 runs, each in a fresh process; AHN5 1 km file (CC BY 4.0). 1 GB = 2^{30} B. Full: results/memory.json.

Take the four rules home

1 Think in arrays, not points

722 961 × 3, as views

slides 6, 11

2 Look before you compute

61 poles on the map, 3 found

slides 19, 26

3 Make it small first

3.0 GB → 0.8 GB peak

slides 13, 26

4 Let compiled code loop

130× faster than a loop

slides 21 to 25

Thank you

To the Python community and to everyone here today.



Sergio Peral
sergioperalgarijo99@gmail.com
github.com/Sloppy1999

Data: AHN5, Het Waterschapshuis, CC BY 4.0.
Licence: creativecommons.org/licenses/by/4.0
[Optional: Title shared with F. Poux, 3D Data Science with Python, O'Reilly 2025; this talk is independent.]